

Python 初級教學

1. 輸出 Hello World 訊息
2. 資料型別
 - 整數 (integer)
 - 浮點數 (float)
 - 布林值 (Boolean)
 - 字串 (string)
3. 變數
4. 列表 (Lists)
5. 字典 (Dictionaries)
6. 條件判斷 (If Statement)
7. 迴圈 (For-Loop)
8. 函式 (Functions)
9. 模組 (Modules)

1. 輸出 Hello World 訊息

```
In [1]: # 輸出我的第一條訊息  
print('Hello World')
```

Hello World

2. 資料型別 (Data Types)

```
In [2]: print(type(1))
```

<class 'int'>

```
In [3]: print(type(2.4))
```

<class 'float'>

```
In [4]: print(1 + 2)
```

3

```
In [5]: print("4-1")
```

4-1

```
In [6]: print(type(True))
```

<class 'bool'>

```
In [7]: print(type(False))
```

<class 'bool'>

```
In [8]: print(type('Hello World'))
```

```
<class 'str'>
```

3. 變數 (Variables)

```
In [9]: message_1 = "I'm learning Python"
print("message_1=", message_1)
```

```
message_1= I'm learning Python
```

```
In [10]: message_2 = "and it's fun"
print(message_2)
```

```
and it's fun
```

```
In [11]: message = message_1 + " " + message_2
print(message)
```

```
I'm learning Python and it's fun
```

```
In [12]: # 另一種格式化輸出的方式
print(f'message_1: {message_1}\nmessage_2: {message_2}')
```

```
message_1: I'm learning Python
```

```
message_2: and it's fun
```

4. 列表 (Lists)

- 新增元素到列表
- 從列表中移除元素
- 訪問列表中的元素
- 列表切片
- 在列表中插入元素
- 合併列表
- 巢狀列表
- 列表排序
- 複製列表

```
In [13]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries)
```

```
# 在列表末尾添加一個元素
countries.append('Japan')
print(countries)
```

```
# 從列表中移除一個元素
countries.remove('Brazil')
print(countries)
```

```
['United States', 'India', 'Taiwan', 'Brazil']
```

```
['United States', 'India', 'Taiwan', 'Brazil', 'Japan']
```

```
['United States', 'India', 'Taiwan', 'Japan']
```

```
In [14]: # 更多移除列表元素的方法

countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries)

# 移除並返回最後一個元素
removed_country = countries.pop()
print(removed_country)

# 移除倒數第一個元素
countries.pop(-1)
print(countries)

# 刪除指定索引的元素
del countries[0]
print(countries)

['United States', 'India', 'Taiwan', 'Brazil']
Brazil
['United States', 'India']
['India']
```

```
In [15]: # 存取列表中的元素 (索引從0開始)
countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[0])
```

United States

```
In [16]: # 使用負數索引從列表末尾開始計數
countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[-1])
```

Brazil

```
In [17]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[-4])
```

United States

```
In [18]: # 列表切片 [起始索引:結束索引]
countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[0:2])
```

['United States', 'India']

```
In [19]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[1:])
```

['India', 'Taiwan', 'Brazil']

```
In [20]: countries = ['United States', 'India', 'Taiwan', 'Brazil']
print(countries[:2])
```

['United States', 'India']

```
In [21]: # 在列表中插入元素
countries = ['United States', 'India', 'Taiwan', 'Brazil']
countries.insert(0, 'Spain')
```

```
print(countries)
```

```
['Spain', 'United States', 'India', 'Taiwan', 'Brazil']
```

```
In [22]: # 列表合併
countries = ['United States', 'India', 'Taiwan', 'Brazil']
countries_2 = ["Finland", "Sweden", "Denmark", "Norway", "Iceland"]
print(countries + countries_2)
```

```
['United States', 'India', 'Taiwan', 'Brazil', 'Finland', 'Sweden',
'Denmark', 'Norway', 'Iceland']
```

```
In [23]: # 巢狀列表 (列表中的列表)
countries = ['United States', 'India', 'Taiwan', 'Brazil']
countries_2 = ['China', 'Japan', 'South Korea', 'Vietnam']
nested_list = [countries, countries_2]
print(nested_list)
```

```
[['United States', 'India', 'Taiwan', 'Brazil'], ['China', 'Japan',
'South Korea', 'Vietnam']]
```

```
In [24]: # 對列表進行排序
numbers = [4, 3, 10, 9, 8, 2]
print(numbers)
numbers.sort()
print(numbers)
```

```
[4, 3, 10, 9, 8, 2]
[2, 3, 4, 8, 9, 10]
```

```
In [25]: # 降序排序列表
numbers = [4, 3, 10, 9, 8, 2]
numbers.sort(reverse=True)
print(numbers)
```

```
[10, 9, 8, 4, 3, 2]
```

```
In [26]: # 更新列表中的元素
numbers = [4, 3, 10, 9, 8, 2]
print(numbers)

numbers[0] = 1000
print(numbers)
```

```
[4, 3, 10, 9, 8, 2]
[1000, 3, 10, 9, 8, 2]
```

```
In [27]: # 複製列表 (使用切片)
countries = ['United States', 'India', 'Taiwan', 'Brazil']
new_list = countries[:]
print(new_list)
```

```
['United States', 'India', 'Taiwan', 'Brazil']
```

```
In [28]: # 另一種複製列表的方法 (使用copy()方法)
new_list2 = countries.copy()
print(new_list2)
```

```
['United States', 'India', 'Taiwan', 'Brazil']
```

5. 字典 (Dictionaries)

1. 複製字典
2. 從字典中移除元素

```
In [29]: my_dict = {'key1': 'value1', 'key2': 'value2'}  
print(my_dict)
```

```
{'key1': 'value1', 'key2': 'value2'}
```

```
In [30]: my_data = {'name': 'Michael', 'age': 63}  
print(my_data)
```

```
{'name': 'Michael', 'age': 63}
```

```
In [31]: print(my_data.keys())
```

```
dict_keys(['name', 'age'])
```

```
In [32]: print(my_data.values())
```

```
dict_values(['Michael', 63])
```

```
In [33]: my_data.items()
```

```
Out[33]: dict_items([('name', 'Michael'), ('age', 63)])
```

```
In [34]: my_data['height'] = 165  
print(my_data)
```

```
{'name': 'Michael', 'age': 63, 'height': 165}
```

```
In [35]: my_data.update({'height': 180})  
print(my_data)
```

```
{'name': 'Michael', 'age': 63, 'height': 180}
```

5.1 Copy a dictionary

```
In [36]: my_data = {'name': 'Michael', 'age': 63, 'height': 165}  
print(my_data)
```

```
new_dict = my_data.copy() # 複製資料  
print(f'new_dict: {new_dict}') # 不會影響到另一個資料副本
```

```
new_dict2 = my_data # 複製參考  
print(f'new_dict2: {new_dict2}') # 指向同一份資料的參考
```

```
{'name': 'Michael', 'age': 63, 'height': 165}  
new_dict: {'name': 'Michael', 'age': 63, 'height': 165}  
new_dict2: {'name': 'Michael', 'age': 63, 'height': 165}
```

```
In [37]: my_data = {'name': 'Michael', 'age': 63, 'height': 165}  
print(f'my_data: {my_data}')
```

```
new_dict = my_data.copy() # 複製資料
```

```
new_dict2 = my_data # 複製參考

my_data.update({'height':180}) # 新增身高資料
print(f'my_data: {my_data}')

print(f'new_dict: {new_dict}') # 不會影響到另一個資料副本
print(f'new_dict2: {new_dict2}') # 指向同一份資料的參考
```

```
my_data: {'name': 'Michael', 'age': 63, 'height': 165}
my_data: {'name': 'Michael', 'age': 63, 'height': 180}
new_dict: {'name': 'Michael', 'age': 63, 'height': 165}
new_dict2: {'name': 'Michael', 'age': 63, 'height': 180}
```

5.2 從字典中移除元素

```
In [38]: my_data = {'name': 'Michael', 'age': 63, 'height': 180}
print(my_data)
my_data.pop('name') # 使用鍵來移除元素
my_data
```

```
{'name': 'Michael', 'age': 63, 'height': 180}
```

```
Out[38]: {'age': 63, 'height': 180}
```

```
In [39]: del my_data['age'] # 使用del刪除指定鍵的元素
my_data
```

```
Out[39]: {'height': 180}
```

```
In [40]: my_data.clear() # 清除字典中所有元素
my_data
```

```
Out[40]: {}
```

6. 條件判斷 (If Statement)

語法 (Syntax):

```
if (條件):
    # 程式區塊
else:
    # 程式區塊
```

```
In [41]: age = 10

if age >= 18:
    print("你是成年人!")
elif age >= 13:
    print("你是青少年。")
else:
    print("你是小孩。")
```

你是小孩。

7. 迴圈 (For-loop)

語法 (Syntax):

```
for i in range(10):  
    print(i)
```

```
In [42]: countries = ['United States', 'India', 'Taiwan', 'Brazil']  
  
countries
```

```
Out[42]: ['United States', 'India', 'Taiwan', 'Brazil']
```

```
In [43]: for country in countries:  
         print(country)
```

```
United States  
India  
Taiwan  
Brazil
```

```
In [44]: for i, country in enumerate(countries):  
         print(i)  
         print(country)
```

```
0  
United States  
1  
India  
2  
Taiwan  
3  
Brazil
```

```
In [45]: my_data = {'name': 'Michael', 'age': 63, 'height': 180}  
  
my_data
```

```
Out[45]: {'name': 'Michael', 'age': 63, 'height': 180}
```

```
In [46]: for key, value in my_data.items():  
         print(key)  
         print(value)
```

```
name  
Michael  
age  
63  
height  
180
```

8. 函式 (Functions)

1. 內建函式 (Built-in functions)

2. 自訂函式 (User-defined Functions)

語法 (Syntax):

```
def 函式名稱(參數):  
    <程式碼>  
    return 回傳值
```

```
In [47]: def add(a, b):  
         return a+b
```

```
In [48]: add(1,3)
```

```
Out[48]: 4
```

8.1 Built-in Functions

```
In [49]: countries = ['United States', 'India', 'Taiwan', 'Brazil']  
         countries
```

```
Out[49]: ['United States', 'India', 'Taiwan', 'Brazil']
```

```
In [50]: len(countries)
```

```
Out[50]: 4
```

```
In [51]: max([10,3, 6, 33, 100])
```

```
Out[51]: 100
```

```
In [52]: min([10,3, 6, 33, 100])
```

```
Out[52]: 3
```

```
In [53]: type(countries)
```

```
Out[53]: list
```

```
In [54]: print('Hello World'.upper())  
         print('Hello World'.lower())  
         print('Hello World'.title())  
         print('Hello World'.count('l'))  
         print('Hello World'.replace('o', '0'))
```

```
HELLO WORLD  
hello world  
Hello World  
3  
Hell0 W0rld
```

8.2 自訂函式 (User-defined Functions)


```
In [55]: import psutil

memory = psutil.virtual_memory()

def convert_bytes(size):
    # 將位元組轉換為GB
    gb = size / (1024 ** 3)
    return gb

total_gb = convert_bytes (memory.total)
available_gb = convert_bytes(memory.available)
used_gb = convert_bytes (memory.used)

print(f"總記憶體: {total_gb:.3f} GB")
print(f"可用記憶體: {available_gb:.3f} GB")
print(f"已用記憶體: {used_gb:.3f} GB")
print(f"記憶體使用率: {memory.percent}%")
```

總記憶體: 16.000 GB
可用記憶體: 5.587 GB
已用記憶體: 7.354 GB
記憶體使用率: 65.1%

9. 模組 (Modules)

1. OS 模組
2. Psutil 模組 (參見上面的自訂函式範例)

語法 (Syntax):

```
import <模組名稱>
```

9.1 OS 模組

```
import os
```

```
In [56]: import os
```

```
In [57]: os.getcwd()
```

```
Out[57]: '/Users/michaelkao/My_Projects/Python'
```

```
In [58]: os.listdir()
```

```
Out[58]: ['graphUSA50.py',
          'my_module.py',
          'hello.py',
          '2. Python Tips and Tricks.pdf',
          'sortDict.py',
          'genExp.py',
          'biTree.py',
          'multiprocessing1.py',
```

```
'process_images_1P.py',
'csvEx2.py',
'xmlEx.py',
'arithmetic.py',
'tkinterEx2.py',
'1. Python Tutorial TW.ipynb',
'randomWalk.py',
'New Folder',
'video',
'gameSquareDays.py',
'unit_test',
'biTree_v1.py',
'2. Python Tips and Tricks.png',
'tupleEx1.py',
'2. Python Tips and Tricks TW.ipynb',
'process_images_CT.py',
'.DS_Store',
'primeLib.py',
'osModule.py',
'smsViaHTTP.py',
'bstEx2.py',
'setEx.py',
'csvEx.py',
'dictEx.py',
'logger',
'gameGuess4.py',
'guiEx.py',
'bsTreeEx.py',
'requirements.txt',
'heapBinary.py',
'uv.lock',
'perfComprehensions.py',
'generatorEx.py',
'employeeEx.py',
'perfHeaps.py',
'loopEx.py',
'process_images_CP.py',
'2. Python Tips and Tricks.html',
'smsViaSMTP.py',
'dunderEx.py',
'test_copy.txt',
'fibo.py',
'bsTree.py',
'loggingEx.py',
'eafp.py',
'jsonEx2.py',
'modEx3.py',
'iterEx2.py',
'xmlEx4.py',
'funcEx.py',
'sysPython.py',
'gameRollDice.py',
'bst.py',
'randomEx.py',
'filesEx6.py',
'graphEx5.py',
```

```
'piEstimate.py',
'gcd.py',
'slicingEx.py',
'gRatio.py',
'stairs.py',
'pyproject.toml',
'filesEx2.py',
'quake.py',
'perf_LvsT.py',
'gameSnake3.py',
'factorialEx.py',
'emailSendHTML.py',
'output',
'sortDict2.py',
'biTreeEx_v2.py',
'emailOnQuake.py',
'multiprocess10.py',
'2. Python Tips and Tricks.ipynb',
'filesEx3.py',
'gameSnake2.py',
'pingSvr.py',
'perfDB.py',
'modEx2.py',
'iterEx3.py',
'__pycache__',
'sortObj.py',
'xmlEx5.py',
'pilLib.py',
'greetings2.py',
'graphEx4.py',
'iterEx.py',
'bstEx.py',
'emailSendAttach.py',
'perfListvsGenerator.py',
'iterEx4.py',
'graphEx.py',
'listEx.py',
'README.md',
'biTreeEx_v1.py',
'xmlEx2.py',
'graphEx3.py',
'gameSRP.py',
'iterEx4_noflag.py',
'greetings.py',
'gameGuessNbr.py',
'SQLiteEx.py',
'jsonEx.py',
'scheduleEx.py',
'4. Python Advanced Topics2.ipynb',
'cacheEx2.py',
'people.py',
'brAsyncEx.py',
'1. Python Tutorial.pdf',
'3. Python Advanced Topics1.ipynb',
'.gitignore',
'heapBinomial.py',
```

```
'gameSnake5.py',
'condFlowEx.py',
'qrCodeGen.py',
'jsonAPI.py',
'.env',
'magicSquare.py',
'my_moduleEx.py',
'cacheEx.py',
'filesEx5.py',
'perfListvsTuple.py',
'graphEx6.py',
'.venv',
'gameSnake4.py',
'iterEx5.py',
'emailPublisher.py',
'modEx4.py',
'sample',
'.python-version',
'qt5sql.py',
'xmlEx3.py',
'brSyncEx.py',
'Python.code-workspace',
'graphEx2.py',
'xmlInspect.py',
'filesEx1.py',
'ProductionPythonCode',
'dates.py',
'go2Mars.py',
'img_processed',
'meminfo.py',
'emailSender.py',
'sortTuple.py',
'qt5Ex1.py',
'httpPing.py',
'gamePoker.py',
'environment.yaml',
'legbEx.py',
'tupleEx3.py',
'comprehensionEx.py',
'fastapiEx.py',
'.git',
'main.py',
'legbEx3.py',
'flagsEx.py',
'numberConversion.py',
'reEx.py',
'glogTime.py',
'biTreeEx.py',
'smsOnQuake.py',
'video-orig',
'comprehensionEx2.py',
'reEmails.py',
'data',
'advFormatting.py',
'csv_processor.log',
'template.env.txt',
```

```
'multiprocessing2.py',  
'legbEx2.py',  
'lambdaEx.py',  
'multiprocessingEx.py',  
'sieveEra.py',  
'photo',  
'1. Python Tutorial.html',  
'tkinterEx1.py',  
'sortList.py',  
'renameEx.py',  
'git',  
'gameSnake.py',  
'1. Python Tutorial.ipynb',  
'mcp_server.py',  
'mem_profile.py',  
'heapFibo.py',  
'daxia_rice_noodle_sign.py',  
'biTree_v2.py',  
'cacheLib.py',  
'tupleEx2.py',  
'exceptionEx.py']
```

```
In [59]: # 建立目錄 (參數：名稱, 模式, 是否允許已存在)  
os.makedirs("New Folder", exist_ok=True)  
os.listdir("./New Folder")
```

```
Out[59]: []
```